Generative and Agentic AI

A Hands-On Handbook

From next-token prediction to safe, multi-agent systems

A self-paced tutorial companion to the 90-day course

Contents

Part I. Generative AI Foundations

Before an agent can act, it needs a brain. This part explains how generative language models work, how to steer them, and how to get reliable, structured output you can build on.

Chapter 1. What Generative AI Is

A generative model produces new content one piece at a time. A language model, the kind we use throughout this book, has learned the statistical shape of human text by reading a very large amount of it. Given some text, it predicts what is likely to come next. Repeat that prediction in a loop and you get sentences, code, and answers.

It is important to be precise about what this is and is not. The model does not look anything up, does not reason in the human sense, and has no memory of you between calls. It is a function from text to a probability distribution over the next token. Everything an agent does is built on top of that one humble capability, combined with tools, memory, and control logic that you supply.

Why this matters

Understanding the model as a next-token predictor demystifies both its strengths and its failures. It explains why prompting works, why the model sometimes invents facts, and why giving it tools and structure is the path to dependable systems.

> **Practice.** Write down three tasks you want an agent to do. For each, note whether it needs fresh information, an action in the world, or only language. That distinction drives the whole design.

Chapter 2. Tokens and Next-Token Prediction

Models do not read characters or whole words. They read tokens, which are common chunks of text. A word like prediction might be one token, while an unusual word may split into several. Costs and context limits are measured in tokens, so it pays to have a feel for them.

At each step the model outputs a score for every possible next token. Those scores become probabilities, one token is chosen, it is appended to the text, and the process repeats. The loop is the entire mechanism of generation.

```
# A toy of the idea: pick the next word by frequency.
import collections, random
text = &amp;quot;the cat sat on the mat the cat ran&amp;quot;.split()
nxt = collections.defaultdict(list)
for a, b in zip(text, text[1:]):
    nxt[a].append(b)
word = &amp;quot;the&amp;quot;
for _ in range(5):
    print(word, end=&amp;quot; &amp;quot;)
    word = random.choice(nxt[word])
```

Practice. Estimate the token count of a paragraph by dividing its character count by about four. Compare with a real tokenizer later to calibrate your intuition.

Chapter 3. Calling a Model

Every later chapter calls a model the same way: send a list of messages, get back a reply. A message has a role, which is system, user, or assistant, and content. The system message sets behavior, the user message asks, and the assistant message is the reply.

Throughout this book a small helper named chat hides provider differences so the same code runs against a hosted model or a free local one. The shape never changes.

```
from shared.llm import chat

reply = chat([
    {&amp;quot;role&amp;quot;: &amp;quot;system&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;You are concise.&amp;quot;},
    {&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;Define an agent in one sentence.&amp;quot;},
])
print(reply)
```

Note. Keep provider keys in environment variables, never in code. The appendix shows the setup for hosted and local models.

Chapter 4. Sampling and Decoding

How the next token is chosen shapes the output. Temperature controls randomness. At zero the model takes the most likely token every time, which is repeatable and good for extraction and code. Higher values add variety, which helps brainstorming but raises the chance of drift.

Top-p, also called nucleus sampling, restricts the choice to the smallest set of tokens whose probabilities add up to p. Together temperature and top-p trade determinism for creativity. For agents that must be reliable, low temperature is usually the right default.

- Temperature 0 to 0.3: extraction, classification, tool use, code.
- Temperature 0.7 to 1.0: drafting, ideation, varied phrasing.

- Set the value deliberately; do not leave it to chance.

Practice. Ask the same question three times at temperature 0, then three times at 1.0. Observe how stability changes.

Chapter 5. Prompting: System Messages and Few-Shot

The system message is your strongest lever. It sets role, tone, format, and rules that persist across the conversation. Specific instructions beat vague ones. Telling the model what to do is better than telling it what not to do.

When a rule is hard to state, show examples instead. A few input and output pairs, called few-shot prompting, teach format and style by demonstration. Two or three good examples often outperform a paragraph of instructions.

```
system = (
    &amp;quot;You classify a message as QUESTION, REQUEST, or COMPLAINT. &amp;quot;
    &amp;quot;Reply with one word only.&amp;quot;
)
examples = [
    {&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;Where is my order?&amp;quot;},
    {&amp;quot;role&amp;quot;: &amp;quot;assistant&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;QUESTION&amp;quot;},
]
```

Practice. Take a prompt that misbehaves and add two worked examples. Measure whether the format becomes reliable.

Chapter 6. Structured Output

Agents need answers a program can read, not prose. Ask the model for JSON that matches a schema, then parse it. Validating the result turns an unpredictable text generator into a dependable component.

Two habits make this robust: state the exact shape you want in the system message, and never trust the output blindly. Strip stray formatting, parse, and validate. If parsing fails, you can ask again or fall back.

```
import json
raw = chat([
    {&amp;quot;role&amp;quot;: &amp;quot;system&amp;quot;, &amp;quot;content&amp;quot;:
        &amp;#x27;Reply ONLY JSON: {&amp;quot;sentiment&amp;quot;: &amp;quot;pos|neg&amp;quot;, &amp;quot;score&amp;quot;: 0..1}&amp;#x27;},
    {&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;I love this product.&amp;quot;},
])
data = json.loads(raw)
print(data[&amp;quot;sentiment&amp;quot;], data[&amp;quot;score&amp;quot;])
```

Note. Libraries such as Pydantic let you declare the schema once and validate automatically. Part III returns to this.

Chapter 7. Embeddings and Similarity

An embedding turns text into a vector of numbers so that similar meanings sit close together. This is the basis of search, recommendation, and retrieval. Closeness is measured with cosine similarity, the cosine of the angle between two vectors.

```
def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = sum(x * x for x in a) ** 0.5
    nb = sum(y * y for y in b) ** 0.5
    return dot / (na * nb)
```

Part VII builds a full retrieval system on this idea. For now, the key insight is that meaning becomes geometry, and geometry is something a computer can compare quickly.

Chapter 8. Limitations and Safety

A language model can state false things confidently, a behavior called hallucination. It can be wrong about recent events, arithmetic, and anything not well represented in its training. It can also be steered by malicious text. Treating output as a draft to be verified, not as truth, is the professional stance.

- Ground answers in retrieved sources and show citations.
- Give the model tools for facts and math rather than trusting it.
- Validate output at the boundary before acting on it.
- Treat any text from tools or the web as untrusted input.

These are not afterthoughts. The strongest agents in this book are the ones with the most disciplined boundaries, and the capstone makes safety a hard gate rather than a hope.

Part II. From Generative to Agentic

An agent is a program that uses a model in a loop to pursue a goal, taking actions through tools and adjusting based on results. This part builds one from first principles so every later framework feels familiar.

Chapter 9. What an Agent Is

A plain model call is a single question and a single answer. An agent wraps that call in a loop: it observes the situation, decides what to do, acts through a tool, observes the result, and repeats until the goal is met. The model supplies the decision; you supply the loop, the tools, and the stopping rule.

This definition is liberating. You do not need a special library to have an agent. You need a model, a few functions it can call, and the discipline to manage the loop. Everything else is convenience.

Chapter 10. The Agent Loop

The loop has four moves: observe, decide, act, repeat. A step budget prevents it from running forever, and a clear finish action lets it stop when done. The skeleton below captures the entire idea in a dozen lines.

```
def agent_loop(goal, tools, max_steps=6):
    context = goal
    for _ in range(max_steps):
        decision = decide(context)          # model picks a tool or finishes
        if decision.action == &amp;quot;finish&amp;quot;:
            return decision.answer
        result = tools[decision.action](decision.arg)
        context += f&amp;quot;\nObservation: {result}&amp;quot;
    return &amp;quot;stopped: out of steps&amp;quot;
```

Practice. Draw the loop for a task you care about. Name the tools and the condition under which the agent should stop.

Chapter 11. Tools as Functions

A tool is just a function the agent may call. Keep tools small, well named, and safe. A calculator that evaluates arithmetic should never run arbitrary code; build it on a parser, not on a blanket evaluation of strings.

```
import ast, operator
OPS = {ast.Add: operator.add, ast.Sub: operator.sub,
       ast.Mult: operator.mul, ast.Div: operator.truediv}

def calc(node):
    if isinstance(node, ast.Constant):
        return node.value
    if isinstance(node, ast.BinOp):
        return OPS[type(node.op)](calc(node.left), calc(node.right))
    raise ValueError(&quot;unsupported&quot;)
```

Note. A safe tool is a security boundary. The calculator above cannot be tricked into executing code, no matter what the model asks.

Chapter 12. Native Tool Calling

Production models have a built-in way to call tools. You pass a list of tool schemas, and instead of plain text the model replies with structured tool calls. You run the tool, return the result, and let it continue. Every framework in Part IV does exactly this under the hood.

```
from shared.llm import chat_with_tools
TOOLS = [{&quot;type&quot;: &quot;function&quot;, &quot;function&quot;: {
    &quot;name&quot;: &quot;calculator&quot;,
    &quot;parameters&quot;: {&quot;type&quot;: &quot;object&quot;,
        &quot;properties&quot;: {&quot;expression&quot;: {&quot;type&quot;: &quot;string&quot;}},
        &quot;required&quot;: [&quot;expression&quot;]}}}]
msg = chat_with_tools([{&quot;role&quot;: &quot;user&quot;, &quot;content&quot;: &quot;7*6?&quot;}], TOOLS)
print(msg.tool_calls)
```

Chapter 13. The ReAct Pattern

ReAct stands for reasoning and acting. The agent writes a short thought, chooses an action, observes the result, and repeats. Making the model emit JSON each turn keeps parsing reliable and makes the reasoning visible, which is a gift when debugging.

```
system = (
  &#x27;Reply each turn with ONLY JSON: &#x27;
  &#x27;{&quot;thought&quot;:&quot;...&quot;,&quot;action&quot;:&quot;calculator|finish&quot;,&quot;action_i
)
```

ReAct is the most influential agent pattern, and you can implement it with nothing more than the plain chat helper. Once you have built it by hand, every framework that offers it will feel like a convenience rather than a mystery.

Chapter 14. Memory

A model has no memory between calls, so the agent must supply it. Short-term memory is a rolling buffer of recent turns. When the buffer grows past a budget, summarize the oldest turns into a running note and keep the most recent ones verbatim. This keeps the agent within the context limit without forgetting the thread.

- Working memory: a scratchpad for the current task, cleared when done.
- Long-term memory: facts that persist across tasks.
- Budgeted memory: summarize the old, keep the recent.

Chapter 15. Reflection

One of the cheapest ways to improve quality is to let the agent check its own work. It drafts an answer, critiques it against the goal, and revises once if needed. This single extra step catches many errors and is the seed of the critic pattern you will scale up with multiple agents later.

```
draft = chat([{&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: goal}])
critique = chat([
    {&amp;quot;role&amp;quot;: &amp;quot;system&amp;quot;, &amp;quot;content&amp;quot;: &amp;quot;Reply OK if good, else one fix.&amp;quot;},
    {&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: f&amp;quot;Goal: {goal}\nAnswer: {draft}&amp;quot;}])
final = draft if critique.startswith(&amp;quot;OK&amp;quot;) else revise(draft, critique)
```

Part III. Tools, Memory, and Planning

A capable agent needs safe typed tools, durable memory, and the ability to plan before it acts. This part hardens the hand-built agent into something you could trust with real work.

Chapter 16. Typed Tools and a Registry

Validate tool inputs before running them. A schema library such as Pydantic lets you declare the expected fields once and reject bad input automatically, while also producing the JSON schema the model needs for native calling. As tools multiply, a registry that maps names to functions becomes the one place to add validation, authorization, and tracing.

```
from pydantic import BaseModel, ValidationError

class WeatherArgs(BaseModel):
    city: str
    units: str = &amp;quot;celsius&amp;quot;

def get_weather(args: dict):
    try:
        a = WeatherArgs(**args)
    except ValidationError as e:
        return f&amp;quot;Error: {e.errors()[0][&amp;#x27;msg&amp;#x27;]}&amp;quot;
    return f&amp;quot;Weather in {a.city}: 21 {a.units}&amp;quot;
```

Chapter 17. Tool Authorization and Least Privilege

The most important security idea for agents is deny by default. An agent identity holds a small set of scopes, and a tool runs only if its required scope is granted. High-impact actions require human approval, and every decision is written to an audit trail. This single control prevents a confused or manipulated agent from doing damage.

```
from shared.policy import PolicyGate, PolicyDenied

gate = PolicyGate(
    tool_scopes={&amp;quot;read_logs&amp;quot;: &amp;quot;logs:read&amp;quot;, &amp;quot;deploy&amp;quot;: &amp;quot;prod:deploy&amp;quot;},
    granted={&amp;quot;logs:read&amp;quot;},          # may read logs, not deploy
    high_blast={&amp;quot;deploy&amp;quot;},
    approver=lambda tool, args: False)

try:
    gate.authorize(&amp;quot;deploy&amp;quot;)
except PolicyDenied as e:
    print(&amp;quot;blocked:&amp;quot;, e)
```

Grant the narrowest set of scopes that still lets the agent do its job. When in doubt, deny and require approval. The capstone in Part IX makes this gate the centerpiece of a real workflow.

Chapter 18. Persistent and Summarized Memory

Long-term memory should survive a restart. A small SQLite table is enough, and it teaches an important habit: always use parameterized queries so user content cannot become SQL. When history grows too large for the context window, summarize the oldest turns and keep the recent ones.

```
import sqlite3
db = sqlite3.connect(&amp;quot;:memory:&amp;quot;)
db.execute(&amp;quot;CREATE TABLE mem (id INTEGER PRIMARY KEY, role TEXT, content TEXT)&amp;quot;)
db.execute(&amp;quot;INSERT INTO mem (role, content) VALUES (?, ?)&amp;quot;,
           (&amp;quot;user&amp;quot;, &amp;quot;remember my name is Sam&amp;quot;))   # parameterized, safe
db.commit()
```

Chapter 19. Planning and Re-Planning

Hard goals become tractable when the agent breaks them into steps first, then executes the steps with results flowing from one to the next. Plans meet reality and break, so a robust agent notices a failed step and asks the planner for a revised plan for the remaining work instead of charging ahead.

```
def plan(goal):
    raw = chat([
        {&amp;quot;role&amp;quot;: &amp;quot;system&amp;quot;, &amp;quot;content&amp;quot;:
            &amp;quot;Return ONLY a JSON array of 2-5 step strings.&amp;quot;},
        {&amp;quot;role&amp;quot;: &amp;quot;user&amp;quot;, &amp;quot;content&amp;quot;: goal}])
    return json.loads(raw)
```

Practice. Give the planner a goal with a step that will fail, and confirm it produces a sensible replacement plan.

Chapter 20. Tracing and Evaluation

You cannot improve what you cannot see or measure. Tracing records one span per tool call with timing, so you can find the slow or expensive step. Evaluation pins behavior with a set of golden cases and a regression gate that fails the build when quality drops. These two practices turn agent development from guesswork into engineering.

```
from shared.tracing import Tracer
tr = Tracer()
with tr.span(&amp;quot;calculator&amp;quot;):
    calculator(&amp;quot;2+2&amp;quot;)
tr.print_trace()
```

Part IV. Agent Frameworks

Having built agents by hand, you can now read any framework, because they all express the same ideas: a loop, tools, memory, and control flow. This part maps your code onto the major options.

Chapter 21. Why Frameworks

Frameworks do not add new concepts; they add names, plumbing, and an ecosystem. Your agent loop becomes a runner or a graph, your tool function becomes a decorated function or a node, and your agent class becomes their agent type. Knowing the mapping means you can move between frameworks without relearning the fundamentals.

- The loop becomes a runner, a graph, or a group chat.
- A tool becomes a decorated function or a plugin.
- Memory becomes a checkpointer or a managed store.
- Handoffs become edges or routing.

Chapter 22. The OpenAI Agents SDK

This SDK packages the loop into an Agent and a Runner, and turns any typed function into a tool with a decorator. Handoffs let one agent route to a more specialized one, which is the first taste of multi-agent design.

```
from agents import Agent, Runner, function_tool

@function_tool
def add(a: int, b: int) -&gt; int:
    return a + b

agent = Agent(name=&quot;Math&quot;, instructions=&quot;Use tools.&quot;, tools=[add])
print(Runner.run_sync(agent, &quot;What is 21 + 21?&quot;).final_output)
```

Chapter 23. LangChain and LangGraph

LangChain composes components with a pipe operator: a prompt feeds a model feeds a parser. LangGraph models an agent as a graph whose nodes transform a shared state and whose edges set the order. Graphs make loops and branching explicit, and a checkpointer gives the graph memory across calls.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    topic: str
    joke: str

def make(state):
    return {&quot;joke&quot;: f&quot;Why did the {state[&#x27;topic&#x27;]} cross the road?&quot;}

g = StateGraph(State)
g.add_node(&quot;make&quot;, make)
g.add_edge(START, &quot;make&quot;); g.add_edge(&quot;make&quot;, END)
print(g.compile().invoke({&quot;topic&quot;: &quot;chicken&quot;})[&quot;joke&quot;])
```

Chapter 24. Semantic Kernel

Semantic Kernel, common in enterprise and .NET settings, treats tools as plugins whose methods carry a decorator. With automatic function calling enabled, the kernel lets the model pick and invoke plugin functions, which is its version of the ReAct loop you built by hand.

```
from semantic_kernel import Kernel
from semantic_kernel.functions import kernel_function

class MathPlugin:
    @kernel_function(name=&quot;add&quot;, description=&quot;Add two integers&quot;)
    def add(self, a: int, b: int) -&gt; str:
        return str(a + b)

kernel = Kernel()
kernel.add_plugin(MathPlugin(), plugin_name=&quot;math&quot;)
```

Chapter 25. The Model Context Protocol

The Model Context Protocol, or MCP, standardizes how agents discover and call tools that live in a separate server. You write a tool once and any MCP-aware client can use it. A minimal server is a few lines, and treating tools as a reusable service is a powerful organizing idea.

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP(&quot;demo&quot;)

@mcp.tool()
def add(a: int, b: int) -&gt; int:
    return a + b

# Serve over stdio with:  mcp.run()
```

Part V. AutoGen

AutoGen frames problem solving as a conversation between agents. This part covers its model, its teams, and a security review team that mirrors the capstone.

Chapter 26. The AutoGen Model

In AutoGen you compose specialized agents that talk to each other to solve a task. The pieces map directly onto what you already know: an assistant agent is your agent class, a tool is a function you pass in, a critic is a second agent, and a human approval step is a user proxy agent. The framework manages the turn taking.

Chapter 27. Agents, Tools, and Teams

The smallest program is one assistant agent backed by a model client. Pass typed functions as tools and the framework exposes them to the model. Put two or more agents in a team and they take turns; an author and a critic, for example, can alternate until the work is approved.

```
from autogen_agentchat.agents import AssistantAgent
from autogen_ext.models.openai import OpenAIChatCompletionClient

client = OpenAIChatCompletionClient(model=&amp;quot;gpt-4o-mini&amp;quot;)
agent = AssistantAgent(&amp;quot;assistant&amp;quot;, model_client=client,
                       system_message=&amp;quot;Be concise.&amp;quot;)
result = await agent.run(task=&amp;quot;Say hello in five words.&amp;quot;)
print(result.messages[-1].content)
```

> *Note. AutoGen calls are asynchronous. In a notebook use top-level await; in a script wrap the call in asyncio.run.*

Chapter 28. Termination and Selection

A team that never stops is a bug. Termination conditions end the chat when a keyword appears or after a maximum number of messages, and conditions combine with simple operators. A round-robin team takes turns in a fixed order, while a selector team lets a manager model choose who speaks next based on context.

```
from autogen_agentchat.conditions import (
    TextMentionTermination, MaxMessageTermination)
term = TextMentionTermination(&amp;quot;APPROVE&amp;quot;) | MaxMessageTermination(6)
```

Chapter 29. A Two-Key Review Team

A powerful pattern pairs a builder that proposes work with a reviewer that must approve it. The reviewer replies APPROVE only when its checks pass, otherwise it names the exact risk to fix. This is the two-key control that the capstone uses to guard image publication, expressed as a conversation.

```
builder = AssistantAgent(&amp;quot;builder&amp;quot;, model_client=client,
    system_message=&amp;quot;Propose a config.&amp;quot;)
reviewer = AssistantAgent(&amp;quot;reviewer&amp;quot;, model_client=client,
    system_message=&amp;quot;Reply APPROVE if safe, else name the risk.&amp;quot;)
```

Part VI. Multi-Agent Orchestration

When one agent is not enough, a team can help, but only when the work has distinct skills or parallel parts. This part covers the patterns that make teams work without runaway cost.

Chapter 30. When to Use Multiple Agents

More agents mean more cost, more latency, and more ways to fail. Reach for a team only when a task has genuinely distinct skills or independent subtasks. Very often a single well-equipped agent with good tools is the better choice. Decide deliberately rather than by fashion.

Chapter 31. Supervisors and Specialists

A supervisor does not do the work; it routes each task to the right specialist. A rule-based router is fast, free, and easy to debug, and you can replace it with a model-based router later. Specialists are focused agents that are simpler to test and reason about than one agent that tries to do everything.

```
def route(task):
    t = task.lower()
    if any(w in t for w in [&amp;quot;add&amp;quot;, &amp;quot;sum&amp;quot;, &amp;quot;+&amp;quot;]):
        return &amp;quot;math&amp;quot;
    if any(w in t for w in [&amp;quot;write&amp;quot;, &amp;quot;draft&amp;quot;]):
        return &amp;quot;writing&amp;quot;
    return &amp;quot;general&amp;quot;
```

Chapter 32. Shared State and Debate

A blackboard is shared memory that every agent can read and write, a simple way to coordinate without direct messaging. A debate pattern pits a proposer against a critic, with a judge to decide, which can beat a single agent that only agrees with itself. Both are extensions of the reflection idea from Part II.

Chapter 33. Parallelism and Budgets

Independent specialists can run at the same time, and a fan-in step merges their outputs. Whatever the topology, a budget that caps steps and tokens is non-negotiable, because a chattering team can run up a bill quickly. Stop cleanly when the budget is spent.

```
from concurrent.futures import ThreadPoolExecutor
def gather(task, specialists):
    with ThreadPoolExecutor() as ex:
        futures = {n: ex.submit(f, task) for n, f in specialists.items()}
        return {n: f.result() for n, f in futures.items()}
```

Part VII. Knowledge and Retrieval-Augmented Generation

To answer from your own knowledge, retrieve relevant text and put it in the prompt. This part builds retrieval from raw vectors to an agent that decides when to search.

Chapter 34. The Retrieval Loop

Retrieval-augmented generation, or RAG, replaces the hope that a model memorized a fact with a reliable process: embed the question, find the most similar chunks of your documents, and put them in the prompt so the answer is grounded. The model generates, but the facts come from your sources.

Chapter 35. Chunking and Embeddings

You cannot embed a whole book as a single vector, so you split it into chunks, with a little overlap so a fact that straddles a boundary still lands in at least one chunk. Each chunk becomes an embedding. Chunk size and overlap are tuning knobs that trade precision against recall.

```
def chunk(text, size=400, overlap=40):
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]
```

Chapter 36. Vector Stores and Retrieval Quality

A vector store keeps text and its embedding together and finds the nearest by cosine similarity. A few lines of code capture the idea that libraries such as Chroma and FAISS scale up. Measure quality with precision and recall: of what you returned, how much was relevant, and of all relevant items, how much did you find.

```
class VectorStore:
    def __init__(self):
        self.items = []
    def add(self, id, text, emb):
        self.items.append((id, text, emb))
    def search(self, qe, k=2):
        scored = sorted(
            ((cosine(qe, e), i, t) for i, t, e in self.items),
            reverse=True)
        return scored[:k]
```

Chapter 37. Citations and Agentic RAG

A grounded answer says where each claim came from, so return source identifiers with the answer and let a reader verify. A naive system always searches; an agentic one lets the model decide whether a question needs retrieval at all, searching for company facts but answering simple questions directly. The decision itself is a small agent.

Part VIII. Production

The difference between a demo and a product is evaluation, observability, guardrails, security, cost control, and a way to serve the agent. This part covers each.

Chapter 38. Evaluation and Continuous Integration Gates

Ship behind a golden set. A harness runs the agent on fixed cases, scores each, and a regression gate fails the build when the mean drops below a threshold. For open-ended answers, a model can act as a grader against stated criteria, used sparingly. A red build is far cheaper than a bad release.

```
from shared.evals import GoldenCase, run_eval, regression_gate
cases = [GoldenCase(&amp;quot;math&amp;quot;, &amp;quot;2+2&amp;quot;, &amp;quot;4&amp;quot;)]
results = run_eval(my_agent, cases, judge=keyword_judge)
regression_gate(results, threshold=0.8)
```

Chapter 39. Tracing and Observability

In production you must see every step. Record spans for each tool call and model call, capture timing and token usage, and export them to a system you can query. The local tracer in this book records the same data that tools such as Langfuse and OpenTelemetry collect at scale.

Chapter 40. Guardrails and Prompt-Injection Defense

Validate at the boundaries. Block disallowed input before it reaches the model and check output before it reaches the user. The most important agent vulnerability is prompt injection, where text returned by a tool or web page instructs the model to misbehave. Treat all such text as untrusted data, scan it, and quarantine anything suspicious.

```
INJECTION = [&amp;quot;ignore previous&amp;quot;, &amp;quot;disregard above&amp;quot;, &amp;quot;system:&amp;quot;]
def is_injection(text):
    low = text.lower()
    return any(p in low for p in INJECTION)

def sanitize(text):
    return &amp;quot;[quarantined]&amp;quot; if is_injection(text) else text
```

Chapter 41. Identity, Budgets, and Serving

Give each agent identity the smallest set of scopes it needs and require approval for high-impact actions, exactly as in Part III. Cap tokens and time with a hard budget. Finally, wrap the agent in an HTTP service so other applications can call it, with typed requests and automatic documentation.

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()

class Query(BaseModel):
    question: str

@app.post(&amp;quot;/chat&amp;quot;)
def chat_endpoint(q: Query):
    return {&amp;quot;answer&amp;quot;: run_agent(q.question)}
```

Part IX. Capstone: An Agentic Image-Build Orchestrator

The capstone brings every idea together in a realistic workflow: an agent that builds a virtual desktop image on a cloud, gated by security checks and a second reviewer that must sign off before anything is published.

Chapter 42. The Orchestrator

The capstone reads a specification, plans the steps to build an image, and runs them through a sequence of tools: create the resource group and identity, define the image, run the build, monitor it, scan it, and only then publish. The orchestrator is the agent loop from Part II, wearing a domain. It uses tracing to show its work and reliability wrappers to survive a flaky step.

Crucially, the tools run in a dry-run mode by default, printing the command they would issue and a simulated result. This makes the whole system safe to run and study, and it mirrors how you should develop any agent that touches real infrastructure: prove the logic before you grant it real power.

Chapter 43. Security Gates and Two-Key Review

Before publishing, the orchestrator scans the image for vulnerable or outdated applications and missing updates. If the scan finds a high-severity issue, publication is blocked. This is a hard gate, not a warning. The example specifications include one compliant image that passes and one vulnerable image that is correctly stopped.

Passing the scan is necessary but not sufficient. A second, independent reviewer must also approve, re-checking the critical posture and adding advisories. Only when both the automated scan and the reviewer agree does the agent proceed, and the highest impact action, updating the host pool, still requires a human. Least privilege, defense in depth, and human oversight, all in one workflow.

If you take one lesson from the capstone, let it be this: the value of an agent is bounded by how well you constrain it. The most trustworthy systems are the ones with the most disciplined gates.

Appendix A. Setup and Providers

The course code runs against a hosted model or a free local one, selected by an environment variable so the same code works everywhere.

A.1 Choosing a Provider

A single helper named chat reads a provider setting and routes to the right backend. Set the provider and its credentials in environment variables. The earliest lessons run with no provider at all, because they teach agent structure with pure Python.

- Hosted model: set the provider to the hosted option and supply an API key.
- Local model: run a local server and point the helper at it; no key or cost.
- Never commit keys; keep them in environment variables or a local file that is ignored by version control.

A.2 Running the Lessons

Each lesson is a notebook with a short concept, a runnable bootstrap cell, an exercise with blanks to fill, and a solution. Run the bootstrap cell first; it puts the shared toolkit on the path. Many lessons in the later parts run entirely offline.

Appendix B. The Shared Toolkit

A small set of reusable modules underpins the whole course, so every lesson speaks the same language.

B.1 Modules

- A model helper that hides provider differences behind one chat function.
- A set of safe example tools, including an arithmetic calculator built on a parser.
- Reliability helpers for retries, timeouts, and circuit breakers.
- A policy gate for deny-by-default tool authorization with an audit trail.
- A tracer that records one span per tool call.
- An evaluation harness with golden cases and a regression gate.

These modules are not magic. Each is small enough to read in a few minutes, and reading them is the fastest way to understand how the lessons fit together. The agent you build by hand and the frameworks you adopt later both rest on the same handful of ideas captured here.